

Modular Multiplication Grokking: Training & Interpretability

This notebook trains a 1-layer transformer on modular multiplication ($a * b \bmod 113$) and analyzes the learned algorithm using both additive and multiplicative Fourier bases.

Pedagogical annotations are included throughout for reproducibility and clarity.

Part 1: Setup & Concepts

1. Imports & Device Setup

What's happening: We import the libraries we need and select the best available hardware.

- **PyTorch** (`torch`): The deep learning framework. Everything is built on "tensors" — multi-dimensional arrays that can live on GPU and track gradients for backpropagation.
- **TransformerLens**: Neel Nanda's library for mechanistic interpretability. It provides `HookedTransformer` — a transformer with hooks at every layer so we can inspect activations.
- **einops**: A library for readable tensor reshaping. Instead of cryptic `.reshape().permute()`, you write `rearrange(x, '(a b) c -> a b c', a=113)` and it's clear what's happening.
- **Device selection**: PyTorch can run on CPU, NVIDIA GPU (CUDA), or Apple GPU (MPS). We pick the best available. MPS is Apple's Metal Performance Shaders — it runs on your Mac's built-in GPU and is 2-5x faster than CPU for this model size.

```
In [1]: import os

# This environment variable tells TransformerLens it's OK to use Apple's MPS
# Without it, TransformerLens shows a warning and may refuse to use MPS.
os.environ["TRANSFORMERLENS_ALLOW_MPS"] = "1"

import torch
import torch.nn.functional as F
import numpy as np
import einops
import tqdm.auto as tqdm
import copy
```

```

from pathlib import Path
import plotly.express as px
import plotly.graph_objects as go

import transformer_lens
import transformer_lens.utils as utils
from transformer_lens import HookedTransformer, HookedTransformerConfig, Act

# --- Device Selection ---
# PyTorch tensors live on a "device". Operations are fastest when tensors are on the same device
# - "cuda" = NVIDIA GPU (Linux/cloud machines)
# - "mps" = Apple Silicon GPU (your Mac)
# - "cpu" = fallback, always works but slower
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
print(f"Using device: {device}")

```

Using device: mps

2. The Task: Modular Multiplication

We're training a transformer to compute $a * b \bmod 113$ for all pairs (a, b) .

Why $p = 113$?

- 113 is **prime**, so every nonzero element has a multiplicative inverse.
- The nonzero integers $\{1, 2, \dots, 112\}$ form a **cyclic group** under multiplication mod 113. This group has order $112 = 2^4 \times 7$, giving it a rich subgroup structure.
- Same prime as Nanda et al., so we can directly compare addition vs multiplication.

Why exclude zero?

For addition, we use all pairs (a, b) with a, b in $\{0, 1, \dots, 112\}$. For multiplication, we **exclude zero** because:

- $0 * b = 0$ for all b — trivially memorizable, no structure to learn
- Zero is not part of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$
- Including it would add noise that obscures the interesting algebraic structure

So our dataset has $112 * 112 = 12,544$ examples (vs. $113^2 = 12,769$ for addition).

How does the transformer see the input?

Each input is a sequence of 3 tokens: $[a, b, =]$

The model predicts the answer at the position of the `=` token. The `=` token is token ID 113 (since integers 0-112 use IDs 0-112).

3. Plotting Helpers

These are convenience wrappers around Plotly. `imshow` shows 2D heatmaps (we'll use it a lot for neuron activation patterns). `line` plots 1D curves (for loss curves, Fourier spectra, etc.).

```
In [2]: def imshow(tensor, renderer=None, xaxis="", yaxis="", **kwargs):
        """Plot a 2D tensor as a heatmap. Uses a diverging red-blue colorscale c
        px.imshow(
            utils.to_numpy(tensor), # Convert from PyTorch tensor to numpy arra
            color_continuous_midpoint=0.0,
            color_continuous_scale="RdBu",
            labels={"x": xaxis, "y": yaxis},
            **kwargs
        ).show(renderer)

def line(tensor, renderer=None, xaxis="", yaxis="", line_labels=None, log_y=
        toggle_x=False, toggle_y=False, return_fig=False, x=None, title="",
        """Plot one or more 1D tensors as line charts."""
        # Handle both a single tensor and a list of tensors
        if isinstance(tensor, list):
            data = [utils.to_numpy(t) for t in tensor]
        else:
            arr = utils.to_numpy(tensor)
            data = [arr] if arr.ndim == 1 else [arr[i] for i in range(arr.shape[

        fig = go.Figure()
        for i, d in enumerate(data):
            name = line_labels[i] if line_labels and i < len(line_labels) else (
            x_vals = utils.to_numpy(x) if x is not None else np.arange(len(d))
            fig.add_trace(go.Scatter(x=x_vals, y=d, name=str(name) if name else

        fig.update_layout(title=title, xaxis_title=xaxis, yaxis_title=yaxis)
        if log_y:
            fig.update_layout(yaxis_type="log")
        if return_fig:
            return fig
        fig.show(renderer)

def scatter(x, y, xaxis="", yaxis="", caxis="", renderer=None, **kwargs):
    px.scatter(y=utils.to_numpy(y), x=utils.to_numpy(x),
               labels={"x": xaxis, "y": yaxis, "color": caxis}, **kwargs).sh
```

Part 2: Data

4. Build the Dataset

We create all $112 * 112 = 12,544$ input-output pairs for $a * b \bmod 113$.

PyTorch concepts introduced:

- `torch.arange(start, end)` creates a 1D tensor `[start, start+1, ..., end-1]`
- `einops.repeat` tiles a tensor along new dimensions — think of it as creating a grid
- `.to(device)` moves a tensor to GPU/MPS memory so computation is faster
- `.long()` ensures the tensor has integer type (int64) — needed for token indices

The grid trick: To get all pairs (a, b) , we repeat `a` along the b -dimension and `b` along the a -dimension, then flatten both to get a list of all pairs.

```
In [3]: p = 113 # Our prime modulus
q = p - 1 # Order of the multiplicative group = 112

# Create all (a, b) pairs with a, b in {1, 2, ..., 112} (exclude zero)
# einops.repeat(torch.arange(1, p), "i -> (i j)", j=q) creates:
# [1,1,1,...,1, 2,2,2,...,2, ..., 112,112,...,112] (each repeated 112 times)
a_vector = einops.repeat(torch.arange(1, p), "i -> (i j)", j=q).long()

# b_vector creates:
# [1,2,3,...,112, 1,2,3,...,112, ..., 1,2,3,...,112] (the sequence repeated q times)
b_vector = einops.repeat(torch.arange(1, p), "j -> (i j)", i=q).long()

# The equals token is always 113 (one past the last integer token)
equals_vector = torch.full((q * q,), p, dtype=torch.long)

# Stack into shape (12544, 3): each row is [a, b, =]
dataset = torch.stack([a_vector, b_vector, equals_vector], dim=1).to(device)

# Labels: a * b mod p
labels = (dataset[:, 0] * dataset[:, 1]) % p

print(f"Dataset shape: {dataset.shape} (should be [12544, 3])")
print(f"Labels shape: {labels.shape} (should be [12544])")
print(f"\nFirst 5 examples:")
for i in range(5):
    a, b = dataset[i, 0].item(), dataset[i, 1].item()
    print(f" {a} * {b} mod {p} = {labels[i].item()}")
```

Dataset shape: torch.Size([12544, 3]) (should be [12544, 3])

Labels shape: torch.Size([12544]) (should be [12544])

First 5 examples:

1 * 1 mod 113 = 1

1 * 2 mod 113 = 2

1 * 3 mod 113 = 3

1 * 4 mod 113 = 4

1 * 5 mod 113 = 5

5. Train/Test Split

We use only **30% for training** and 70% for testing. This extreme split is essential for grokking:

- With too much training data, the model generalizes quickly (no delayed "aha" moment)
- With too little, it may never generalize at all
- 30% is the sweet spot found by Power et al. (2022)

Why grokking needs this: The model first memorizes all ~3,763 training examples (which is easy for a ~400K parameter model). Only after continued training with weight decay does it discover the generalizing algorithm that works on the unseen 70%.

```
In [4]: DATA_SEED = 598 # Same seed as Nanda's notebook for reproducibility
        frac_train = 0.3

        # torch.manual_seed makes the random permutation reproducible
        torch.manual_seed(DATA_SEED)

        # Randomly shuffle the indices and split
        indices = torch.randperm(q * q) # Random permutation of [0, 1, ..., 12543]
        cutoff = int(q * q * frac_train)

        train_indices = indices[:cutoff]
        test_indices = indices[cutoff:]

        train_data = dataset[train_indices]
        train_labels = labels[train_indices]
        test_data = dataset[test_indices]
        test_labels = labels[test_indices]

        print(f"Training set: {train_data.shape[0]} examples ({frac_train*100:.0f}%")
        print(f"Test set: {test_data.shape[0]} examples ({(1-frac_train)*100:.0f}%")
```

Training set: 3763 examples (30%)

Test set: 8781 examples (70%)

Part 3: Model

6. Building the Transformer

We use TransformerLens's `HookedTransformerConfig` to define the architecture, then `HookedTransformer` to build it. Here's what each parameter means:

Parameter	Value	What it does
<code>n_layers</code>	1	Number of transformer blocks. We use 1 — the simplest possible.
<code>n_heads</code>	4	Number of attention heads. Each head can learn a different attention pattern.
<code>d_model</code>	128	Width of the "residual stream" — the main highway of information through the model.
<code>d_head</code>	32	Dimension per attention head ($128 / 4 = 32$).
<code>d_mlp</code>	512	Width of the MLP hidden layer (4x expansion). This is where the core computation happens.
<code>act_fn</code>	"relu"	MLP activation function: $\max(0, x)$. Introduces nonlinearity.
<code>normalization_type</code>	None	No LayerNorm — removed for interpretability (one less thing to analyze).
<code>d_vocab</code>	114	Input vocabulary: tokens 0-112 (integers) + token 113 (equals sign).
<code>d_vocab_out</code>	113	Output vocabulary: the model predicts one of 113 possible answers (0-112).
<code>n_ctx</code>	3	Context length: each input is 3 tokens [a, b, =].

The Architecture in Plain English

```
Input: [a, b, =] (3 integer tokens)
  |
  v
Token Embedding (W_E)      Each token -> 128-dim vector
(learned lookup table)
  +
Position Embedding (W_pos) Each position (0,1,2) -> 128-dim
vector
  |
  v
Attention Layer             4 heads look at all positions,
route information to '=' position
  + (residual connection)
  v
MLP Layer                  512 neurons with ReLU. THIS is
where the math happens.
```

```

    + (residual connection)
    v
Unembedding (W_U)          128-dim vector -> 113 logits (one
per possible answer)
    |
    v
Prediction: argmax of logits = the model's answer

```

Total parameters: ~400,000 (tiny by modern standards — GPT-2 has 117 million).

```

In [5]: cfg = HookedTransformerConfig(
    n_layers=1,
    n_heads=4,
    d_model=128,
    d_head=32,
    d_mlp=512,
    act_fn="relu",
    normalization_type=None,      # No LayerNorm — easier to interpret
    d_vocab=p + 1,               # 0-112 + equals token
    d_vocab_out=p,               # Predict one of 113 answers
    n_ctx=3,                     # Input is [a, b, =]
    init_weights=True,
    device=device,
    seed=42,                     # Different seed than addition (999) — indepe
)

model = HookedTransformer(cfg)
print(f"Model has {sum(p.numel() for p in model.parameters()):,} parameters")

```

Model has 227,313 parameters

Disable biases: We freeze all bias parameters (set `requires_grad = False`). This doesn't hurt performance for this task and makes the model easier to interpret — every computation is a clean matrix multiply without additive offsets.

```

In [6]: for name, param in model.named_parameters():
    if "b_" in name: # All bias parameters in TransformerLens start with "b_"
        param.requires_grad = False

# Count trainable vs frozen parameters
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
frozen = sum(p.numel() for p in model.parameters() if not p.requires_grad)
print(f"Trainable: {trainable:,} | Frozen (biases): {frozen:,}")

```

Trainable: 226,048 | Frozen (biases): 1,265

7. The Residual Stream View

The key insight for mechanistic interpretability is the **residual stream decomposition**. Because of residual connections (the `+` in the architecture above), the final output is a **sum** of contributions:

$$\text{logits} = \underbrace{(\text{embedding}) @ W_U}_{\text{MLP contribution}} + \underbrace{(\text{attention output}) @ W_U}_{\text{attention contribution}} + \underbrace{(\text{MLP output}) @ W_U}_{\text{direct path}}$$

Each term can be analyzed independently. For modular addition, Nanda et al. found:

- The **direct path** contributes almost nothing
- **Attention** mainly routes information (copies embeddings of a and b to the = position)
- The **MLP** does the actual computation (implements the trig identity for addition)

A central question for our project: **does multiplication follow the same pattern, or does attention play a more active computational role?**

Part 4: Training

8. Optimizer & Loss Function

AdamW Optimizer

AdamW is the standard optimizer for transformers. The key parameters:

- **Learning rate** (`lr = 1e-3`): How big each gradient step is. 0.001 is standard.
- **Weight decay** (`wd = 1.0`): This is **crucial for grokking**. Every step, each weight is multiplied by $(1 - lr * wd) = 0.999$. This slowly shrinks all weights toward zero. Without it, the model memorizes and never generalizes. With it, memorized circuits (which need large, specific weights) get eroded, while the generalizing circuit (which is simpler/lower-norm) survives.
- **Betas** (`(0.9, 0.98)`): Momentum parameters. beta1 controls how much the gradient direction is smoothed, beta2 controls how much the gradient magnitude is smoothed.

Loss Function: Cross-Entropy

The model outputs 113 "logits" (raw scores, one per possible answer). We convert to probabilities via softmax, then measure how much probability the model puts on the correct answer. Lower loss = more confident and correct.

- `log_softmax` : Numerically stable way to compute $\log(\text{softmax}(x))$

- `.gather(dim=-1, index=labels)` : Picks out the log-probability of the correct answer
- Uniform loss (random guessing) = $-\log(1/113) = \log(113) \approx 4.73$

```
In [7]: # Training hyperparameters (same as Nanda et al.)
lr = 1e-3
wd = 1.0
betas = (0.9, 0.98)
num_epochs = 40000          # More than addition (25K) – multiplication may take
checkpoint_every = 100

# Create optimizer
# model.parameters() returns all trainable weights – the optimizer updates them
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=wd, betas=betas)

def loss_fn(logits, labels):
    """
    Cross-entropy loss for training. Uses float32 (MPS compatible).
    Gradients flow through this – used in the training loop for .backward().
    """
    if len(logits.shape) == 3:
        logits = logits[:, -1]
    logits = logits.to(torch.float32)
    log_probs = logits.log_softmax(dim=-1)
    correct_log_probs = log_probs.gather(dim=-1, index=labels[:, None])[:, 0]
    return -correct_log_probs.mean()

def loss_fn_precise(logits, labels):
    """
    High-precision loss for LOGGING only (not training). Moves to CPU first,
    then casts to float64 to capture micro-scale oscillations in the loss landscape.
    Do NOT use this for .backward() – it detaches from the computation graph.
    """
    if len(logits.shape) == 3:
        logits = logits[:, -1]
    # MPS cannot cast directly to float64 – move to CPU first, then cast
    logits = logits.detach().cpu().to(torch.float64)
    labels_cpu = labels.detach().cpu()
    log_probs = logits.log_softmax(dim=-1)
    correct_log_probs = log_probs.gather(dim=-1, index=labels_cpu[:, None])[:, 0]
    return -correct_log_probs.mean().item()

# Sanity check: initial loss should be near uniform (log(113) = 4.73)
with torch.inference_mode():
    init_logits = model(train_data)
    init_loss_f32 = loss_fn(init_logits, train_labels)
    init_loss_f64 = loss_fn_precise(init_logits, train_labels)
    print(f"Initial loss (float32): {init_loss_f32.item():.6f}")
    print(f"Initial loss (float64): {init_loss_f64:.10f}")
    print(f"Uniform loss: {np.log(p):.10f}")
```

```
Initial loss (float32): 4.736202
Initial loss (float64): 4.7362016400
Uniform loss:          4.7273878187
```

9. Training Loop

Full-batch training: We pass ALL training examples in every step (not random mini-batches). This is unusual — most deep learning uses stochastic gradient descent (SGD) with mini-batches. Nanda uses full-batch because:

1. The dataset is small enough (3,763 examples) to fit in memory
2. Full-batch makes training smoother (no noise from random sampling)
3. It avoids "slingshot" dynamics that can disrupt grokking

We save a complete copy of the model every 100 epochs so we can analyze how the internal representations evolve over training.

```
In [8]: # Toggle this to False after the first successful training run.
# Then you can restart the kernel and skip straight to analysis.
TRAIN_MODEL = False
```

```
PTH_LOCATION = "./checkpoints/multiplication_grokking.pth"
os.makedirs(Path(PTH_LOCATION).parent, exist_ok=True)
```

```
In [9]: train_losses = []
test_losses = []
model_checkpoints = []
checkpoint_epochs = []

if TRAIN_MODEL:
    for epoch in tqdm.tqdm(range(num_epochs)):
        # --- Forward pass ---
        train_logits = model(train_data)
        train_loss = loss_fn(train_logits, train_labels) # float32 for back

        # --- Backward pass ---
        train_loss.backward()

        # Log with float64 precision (detached, on CPU – doesn't affect train)
        train_losses.append(loss_fn_precise(train_logits, train_labels))

        # --- Update weights ---
        optimizer.step()
        optimizer.zero_grad()

        # --- Evaluate on test set (no gradients needed) ---
        with torch.inference_mode():
            test_logits = model(test_data)
            test_losses.append(loss_fn_precise(test_logits, test_labels))

        # --- Save checkpoint ---
        if ((epoch + 1) % checkpoint_every) == 0:
```

```

        checkpoint_epochs.append(epoch)
        model_checkpoints.append(copy.deepcopy(model.state_dict()))
        if (epoch + 1) % 5000 == 0:
            print(f"Epoch {epoch+1:>5d} | Train: {train_losses[-1]:.10f}")

    print(f"\nTraining complete! Final train loss: {train_losses[-1]:.10f},

```

In [10]: *# Save everything to disk (only if we actually trained)*

```

if TRAIN_MODEL and len(train_losses) > 0:
    torch.save({
        "model": model.state_dict(),
        "config": model.cfg,
        "checkpoints": model_checkpoints,
        "checkpoint_epochs": checkpoint_epochs,
        "test_losses": test_losses,
        "train_losses": train_losses,
        "train_indices": train_indices,
        "test_indices": test_indices,
    }, PTH_LOCATION)
    print(f"Saved {len(train_losses)} epochs to {PTH_LOCATION}")

```

In [11]: *# Reload from disk (when TRAIN_MODEL = False)*

```

if not TRAIN_MODEL:
    cached_data = torch.load(PTH_LOCATION, map_location=device, weights_only)
    model.load_state_dict(cached_data["model"])
    model_checkpoints = cached_data["checkpoints"]
    checkpoint_epochs = cached_data["checkpoint_epochs"]
    test_losses = cached_data["test_losses"]
    train_losses = cached_data["train_losses"]
    train_indices = cached_data["train_indices"]
    test_indices = cached_data["test_indices"]
    print(f"Loaded {len(model_checkpoints)} checkpoints from {PTH_LOCATION}")

```

Loaded 40 checkpoints from ./checkpoints/multiplication_grokking.pth

10. Loss Curves — Did It Grok?

The signature of grokking:

1. **Train loss drops fast** (first ~1000 epochs) — the model memorizes the training data
2. **Test loss stays high** for a long time — memorization doesn't generalize
3. **Test loss suddenly drops** — the model discovers the generalizing algorithm

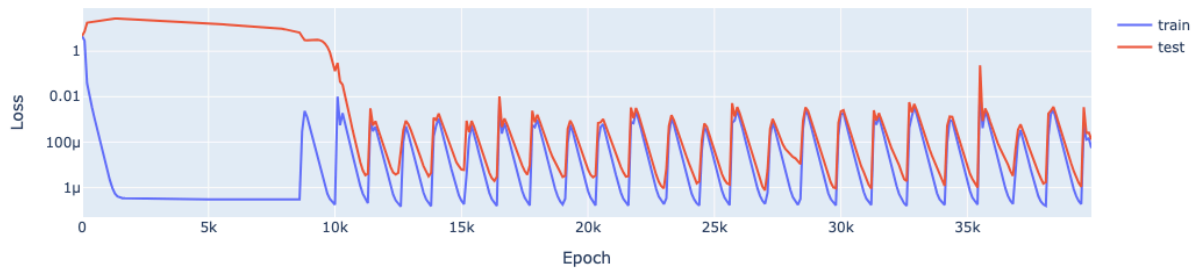
If you don't see the test loss drop, the model may need more epochs.

```

In [12]: line(
    [train_losses[::100], test_losses[::100]],
    x=np.arange(0, len(train_losses), 100),
    xaxis="Epoch", yaxis="Loss",
    log_y=True,
    title="Multiplication: Training Curve (Grokking?)",
    line_labels=["train", "test"]
)

```

Multiplication: Training Curve (Grokking?)



Part 5: Initial Analysis

11. Embedding Analysis: Additive Fourier Basis

For **addition**, Nanda showed the embedding W_E has a **sparse** Fourier spectrum: only ~5-6 key frequencies have significant energy. The model learned to represent each number as a point on a circle (the Clock algorithm).

Here we apply the same analysis to multiplication. We construct the **additive** Fourier basis (sin and cos at each frequency k for the integers 1 to 112), project the embedding onto this basis, and plot the norms.

Prediction: The spectrum will look **dense/flat** — energy spread across all frequencies. This is what Doshi et al. (2024) and Furuta et al. (2024) found. But it might be because we're using the wrong basis! (We'll test the multiplicative basis later.)

What is a Fourier basis?

It's a set of sine and cosine waves at different frequencies. Any function on $\{1, \dots, 112\}$ can be decomposed as a weighted sum of these waves. If only a few frequencies have large weights, the function has a simple periodic structure. If all frequencies are needed, the function is complex/non-periodic.

```
In [13]: # Run the model on ALL inputs and cache every intermediate activation
# run_with_cache is the key TransformerLens feature – it records everything
original_logits, cache = model.run_with_cache(dataset)

# Extract key weight matrices
# W_E[:-1] excludes the equals token embedding (we only care about integer t
# But for multiplication, we want tokens 1-112 (indices 1 through p-1)
W_E = model.embed.W_E[1:p] # Shape: (112, 128) – one 128-dim vector per int
print(f"W_E shape: {W_E.shape} (should be [112, 128])")
```

```
W_logit = model.blocks[0].mlp.W_out @ model.unembed.W_U
print(f"W_logit shape: {W_logit.shape} (512 neurons -> 113 output logits)")
```

W_E shape: torch.Size([112, 128]) (should be [112, 128])

W_logit shape: torch.Size([512, 113]) (512 neurons -> 113 output logits)

```
In [14]: # Build the additive Fourier basis for {1, 2, ..., 112}
# This is the "standard" Fourier transform – it's the natural basis for ADD1
# For multiplication, we may need a different basis (the multiplicative char

fourier_basis = []
fourier_basis_names = []

# Constant component (DC term)
fourier_basis.append(torch.ones(q, dtype=torch.float32))
fourier_basis_names.append("Constant")

# Sin and cos at each frequency k = 1, 2, ..., 56
for freq in range(1, q // 2 + 1):
    # Note: we index over {1,...,112} so we use torch.arange(1, p)
    x = torch.arange(1, p, dtype=torch.float32)
    fourier_basis.append(torch.sin(2 * torch.pi * freq * x / q))
    fourier_basis_names.append(f"Sin {freq}")
    fourier_basis.append(torch.cos(2 * torch.pi * freq * x / q))
    fourier_basis_names.append(f"Cos {freq}")

# Stack into a matrix and normalize each row to unit length
fourier_basis = torch.stack(fourier_basis, dim=0).to(torch.float32).to(device)
fourier_basis = fourier_basis / fourier_basis.norm(dim=-1, keepdim=True)

print(f"Fourier basis shape: {fourier_basis.shape} ({len(fourier_basis_names)} components x {q} integers)")
print(f"Dtype: {fourier_basis.dtype}")

# Verify orthogonality: the dot product of any two different basis vectors should be 0
imshow(fourier_basis @ fourier_basis.T, title="Fourier Basis Orthogonality (should be ~identity)")
```

Fourier basis shape: torch.Size([113, 112]) (113 components x 112 integers)

Dtype: torch.float32

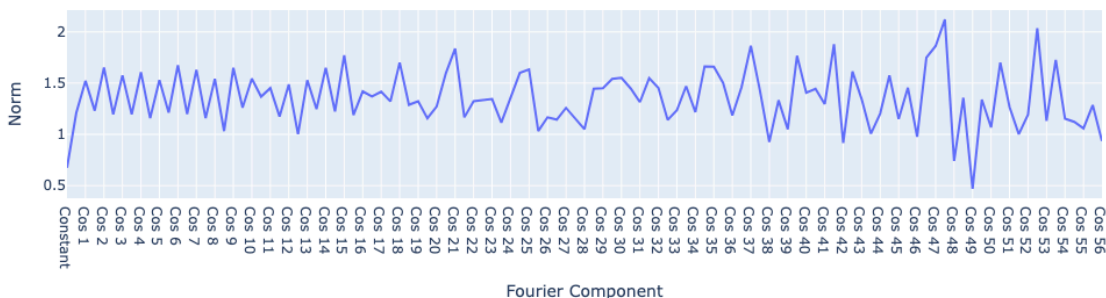
Fourier Basis Orthogonality (should be ~identity)



```
In [15]: # Project the embedding into the Fourier basis
# fourier_basis @ W_E has shape (113 components, 128 residual stream dims)
# The norm of each row tells us how much energy that frequency has in the embedding
fourier_embed = fourier_basis @ W_E
```

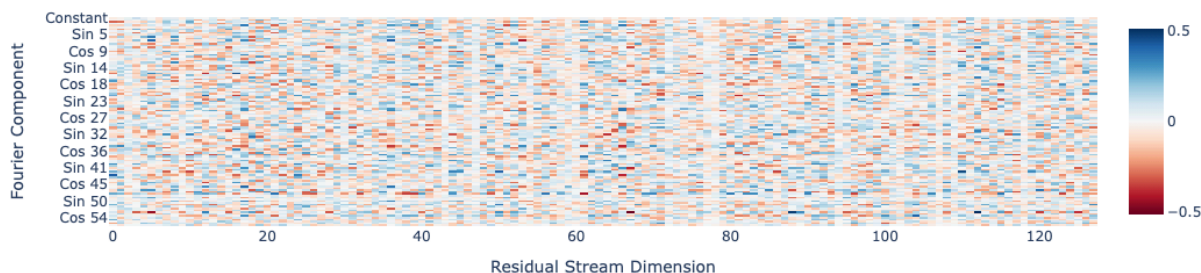
```
fourier_norms = fourier_embed.norm(dim=-1) # Shape: (113,) – one norm per f
line(
    fourier_norms,
    x=fourier_basis_names,
    xaxis="Fourier Component",
    yaxis="Norm",
    title="Embedding Fourier Spectrum (Additive Basis) – Is It Sparse or Dense"
)
```

Embedding Fourier Spectrum (Additive Basis) — Is It Sparse or Dense?



```
In [16]: # Full Fourier embedding heatmap (each row = one Fourier component, each col
imshow(
    fourier_embed,
    yaxis="Fourier Component",
    xaxis="Residual Stream Dimension",
    y=fourier_basis_names,
    title="Embedding in Fourier Basis (rows = frequencies, columns = residual stream)"
)
```

Embedding in Fourier Basis (rows = frequencies, columns = residual stream)



12. Neuron Activation Heatmaps

Each MLP neuron has an activation for every input pair (a, b). We can reshape these into a 112x112 grid and visualize as a heatmap.

For addition neurons: Nanda showed these look like $\cos(ka) * \cos(kb)$ — clear periodic stripes at a single frequency.

For multiplication neurons: We expect something different. If the neuron is doing computation related to $a \cdot b \bmod p$, the pattern should reflect the multiplication table's structure — which is much more complex than addition's simple diagonal stripes.

What to look for:

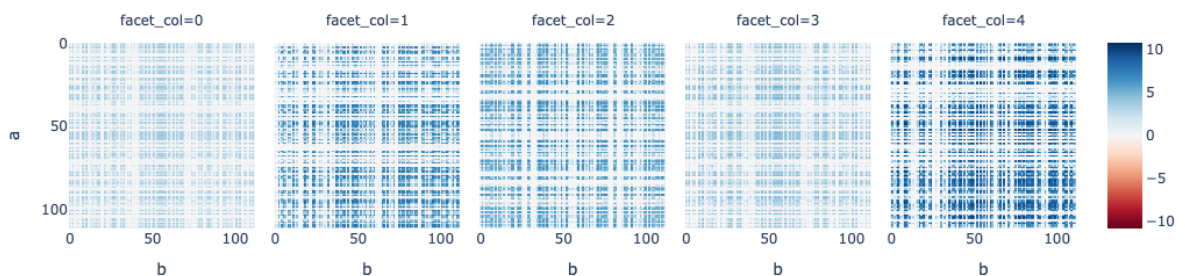
- Clear periodic structure? -> some Fourier/character structure exists
- Random-looking noise? -> the neuron might be memorizing, or we need a different basis
- Block structure? -> possible coset/subgroup decomposition (H7)

```
In [17]: # Extract neuron activations at the '=' position (the last token, where prec
# cache["post", 0, "mlp"] = activations AFTER ReLU in layer 0's MLP
# Shape: (12544, 3, 512) -> we take[:, -1, :] for the '=' position -> (12544
neuron_acts = cache["post", 0, "mlp"][:, -1, :]
print(f"Neuron activations shape: {neuron_acts.shape} (12544 inputs x 512 r

# Reshape first 5 neurons to 112x112 grids and plot
# einops.rearrange: "(a b) neuron -> neuron a b" unfolds the flat 12544 back
imshow(
    einops.rearrange(neuron_acts[:, :5], "(a b) neuron -> neuron a b", a=q,
    title="First 5 MLP Neuron Activations (each panel = one neuron)",
    xaxis="b", yaxis="a",
    facet_col=0 # Put each neuron in its own subplot
)
```

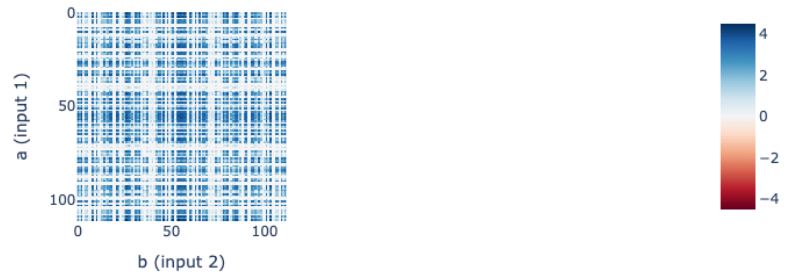
Neuron activations shape: torch.Size([12544, 512]) (12544 inputs x 512 neurons)

First 5 MLP Neuron Activations (each panel = one neuron)



```
In [18]: # Zoom in on a single neuron
imshow(
    einops.rearrange(neuron_acts[:, 0], "(a b) -> a b", a=q, b=q),
    title="Neuron 0 Activation Heatmap: h_0(a, b)",
    xaxis="b (input 2)",
    yaxis="a (input 1)"
)
```

Neuron 0 Activation Heatmap: $h_0(a, b)$



13. Attention Patterns

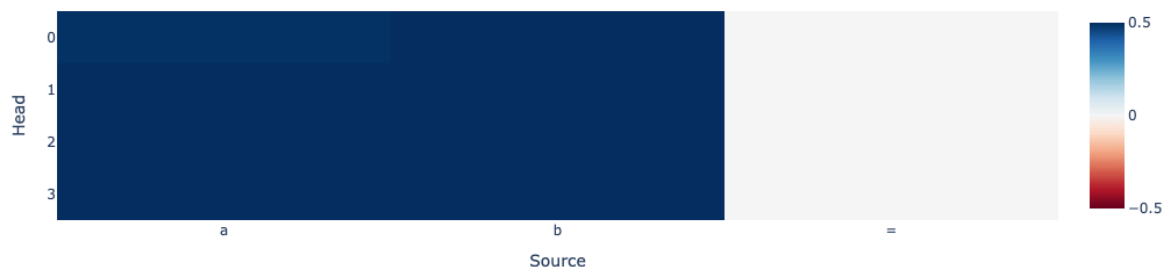
The attention layer decides how much each position "looks at" other positions. For the `=` position (where we predict), the attention weights tell us how much information from `a` and `b` is routed to the output.

For addition: Attention is mostly uniform — each head just copies the embeddings of `a` and `b` to the `=` position. The attention weights don't depend on the actual values of `a` and `b`.

For multiplication (H5): If attention plays a computational role, the attention weights should depend on the *values* of `a` and `b` (not just their positions).

```
In [19]: # Average attention pattern across all inputs
# cache["pattern", 0] has shape (12544, 4 heads, 3 positions, 3 positions)
# We look at what the '=' position (index -1) attends to
avg_pattern = cache["pattern", 0].mean(dim=0)[: , -1, :] # Shape: (4 heads,
imshow(
    avg_pattern,
    title="Average Attention from '=' Position (per head)",
    xaxis="Source", yaxis="Head",
    x=["a", "b", "="]
)
```

Average Attention from '=' Position (per head)




```
In [20]: # Check if attention is input-dependent: how much does the attention to 'a'
# across different input values?
# If attention is pure routing, this should be ~0 (same pattern for all inputs)
# If attention is computational, this should be significant
attn_to_a = cache["pattern", 0][:, :, -1, 0] # Shape: (12544, 4) - attention weights to 'a'
attn_to_b = cache["pattern", 0][:, :, -1, 1] # Shape: (12544, 4) - attention weights to 'b'

print("Attention to 'a' - std across inputs per head (higher = more input-dependent):")
for h in range(4):
    print(f"Head {h}: std = {attn_to_a[:, h].std().item():.4f}, "
          f"mean = {attn_to_a[:, h].mean().item():.4f}")

print("\nAttention to 'b' - std across inputs per head:")
for h in range(4):
    print(f"Head {h}: std = {attn_to_b[:, h].std().item():.4f}, "
          f"mean = {attn_to_b[:, h].mean().item():.4f}")
```

Attention to 'a' - std across inputs per head (higher = more input-dependent):

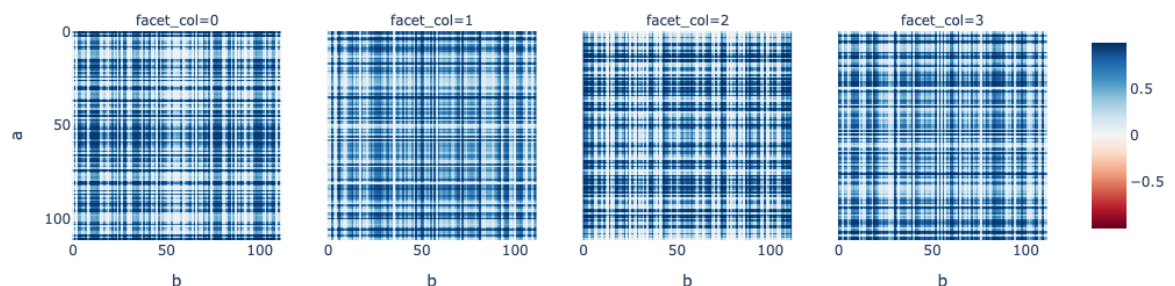
```
Head 0: std = 0.3382, mean = 0.4972
Head 1: std = 0.2914, mean = 0.4991
Head 2: std = 0.3400, mean = 0.5002
Head 3: std = 0.3149, mean = 0.4985
```

Attention to 'b' - std across inputs per head:

```
Head 0: std = 0.3382, mean = 0.5012
Head 1: std = 0.2914, mean = 0.4990
Head 2: std = 0.3400, mean = 0.4988
Head 3: std = 0.3149, mean = 0.5003
```

```
In [21]: # Visualize attention-to-a as a function of (a, b) for each head
# If attention is input-independent, these heatmaps will be uniform (flat color)
# If input-dependent, we'll see structure
imshow(
    einops.rearrange(attn_to_a, "(a b) head -> head a b", a=q, b=q),
    title="Attention weight from '=' to 'a' (per head, as function of input)",
    xaxis="b", yaxis="a", facet_col=0
)
```

Attention weight from '=' to 'a' (per head, as function of input)



Part 6: The Decisive Experiment

14. The "Wrong Basis" Hypothesis

The additive Fourier spectrum (Section 11) appears dense for multiplication. But this may be because we analyzed in the **wrong basis**.

The additive Fourier basis uses $\sin(2\pi ka/112)$ and $\cos(2\pi ka/112)$, which is the natural basis for **addition** (periodic in the integer a). For **multiplication**, the natural basis uses the **multiplicative characters**: $\sin(2\pi k \log_g(a)/112)$ and $\cos(2\pi k \log_g(a)/112)$, where $\log_g(a)$ is the discrete logarithm.

The core isomorphism: multiplication in $(\mathbb{Z}/113\mathbb{Z})^*$ is isomorphic to addition in $\mathbb{Z}/112\mathbb{Z}$ via the discrete logarithm. If the model exploits this isomorphism, its embedding should be sparse in the multiplicative basis even though it looks dense in the additive basis.

This is the single most informative experiment for the project. No prior work has applied the multiplicative character basis to a trained multiplication transformer.

Primitive Roots and the Discrete Logarithm

Since 113 is prime, the nonzero integers $\{1, 2, \dots, 112\}$ form a **cyclic group** under multiplication mod 113. A **primitive root** g is a generator: every nonzero integer a can be written as $a = g^\alpha \bmod 113$ for a unique α in $\{0, 1, \dots, 111\}$. The map $a \rightarrow \alpha$ is the **discrete logarithm**: $\alpha = \log_g(a)$.

The key property: $\log_g(a * b \bmod 113) = (\log_g(a) + \log_g(b)) \bmod 112$. Multiplication becomes addition in log-space.

For $p = 113$, the smallest primitive root is $g = 3$.

```
In [22]: # Find primitive root of p=113 and build discrete log table
g = 3 # smallest primitive root of 113

# Verify: g must have order p-1 = 112
val, order = g, 1
while val != 1:
    val = (val * g) % p
    order += 1
assert order == p - 1, f"g={g} has order {order}, not {p-1}!"
print(f"Primitive root of {p}: g = {g} (verified: order = {order})")

# Build discrete log table: dlog[a] = alpha where a = g^alpha mod p
dlog = {}
val = 1
for alpha in range(p):
```

```

    dlog[val] = alpha
    val = (val * g) % p

# log_vals[i] = log_g(i+1) for i=0,...,111
# This maps integers 1,...,112 (in order) to their discrete logs
log_vals = torch.tensor([dlog[a] for a in range(1, p)], dtype=torch.float32)

# Sanity checks
print(f"\nExamples:")
print(f"    log_3(1) = {dlog[1]} (should be 0)")
print(f"    log_3(3) = {dlog[3]} (should be 1)")
print(f"    log_3(9) = {dlog[9]} (should be 2, since 3^2=9)")

# Verify the isomorphism
a_test, b_test = 7, 15
prod = (a_test * b_test) % p
assert dlog[prod] == (dlog[a_test] + dlog[b_test]) % q
print(f"\n    log_3({a_test} * {b_test} mod {p}) = log_3({prod}) = {dlog[prod]}")
print(f"    log_3({a_test}) + log_3({b_test}) mod {q} = {dlog[a_test]} + {dlog[b_test]}")
print(f"    Isomorphism verified!")

```

Primitive root of 113: $g = 3$ (verified: order = 112)

Examples:

```

log_3(1) = 0 (should be 0)
log_3(3) = 1 (should be 1)
log_3(9) = 2 (should be 2, since 3^2=9)

```

```

log_3(7 * 15 mod 113) = log_3(105) = 92
log_3(7) + log_3(15) mod 112 = 8 + 84 mod 112 = 92
Isomorphism verified!

```

Constructing the Multiplicative Character Basis

The multiplicative basis has the **same shape** as the additive basis: (113, 112). Each row is a basis function evaluated at $a = 1, 2, \dots, 112$.

The only difference: where the additive basis uses a as the argument to $\sin/\cos$, the multiplicative basis uses $\log_g(a)$.

- Additive: $\sin(2\pi k \cdot a / 112)$
- Multiplicative: $\sin(2\pi k \cdot \log_g(a) / 112)$

Since $\log_g$ is a bijection on $\{1, \dots, 112\}$, the inner products reduce to standard Fourier orthogonality, so the basis is still orthogonal.

In [23]:

```

# Build the multiplicative character basis
# Same structure as the additive fourier_basis, but using log_g(a) instead of a

mult_basis = []
mult_basis_names = []

# Constant component
mult_basis.append(torch.ones(q, dtype=torch.float32))

```

```

mult_basis_names.append("Constant")

# Sin and cos at each frequency k = 1, ..., 56
for freq in range(1, q // 2 + 1):
    # log_vals[i] = log_g(i+1), same ordering as the columns of W_E
    mult_basis.append(torch.sin(2 * torch.pi * freq * log_vals / q))
    mult_basis_names.append(f"Sin {freq}")
    mult_basis.append(torch.cos(2 * torch.pi * freq * log_vals / q))
    mult_basis_names.append(f"Cos {freq}")

mult_basis = torch.stack(mult_basis, dim=0).to(torch.float32).to(device)
# Normalize, clamping to avoid division by zero for the degenerate sin(56) r
mult_basis = mult_basis / mult_basis.norm(dim=-1, keepdim=True).clamp(min=1e-6)

print(f"Multiplicative basis shape: {mult_basis.shape} (should match additive

# VISUAL CHECK: orthogonality heatmap should look like the identity matrix
# One row (Sin 56) may be degenerate (all zeros) -- that's expected
imshow(mult_basis @ mult_basis.T, title="Multiplicative Basis Orthogonality

```

Multiplicative basis shape: torch.Size([113, 112]) (should match additive: torch.Size([113, 112]))

Multiplicative Basis Orthogonality (should be ~identity)



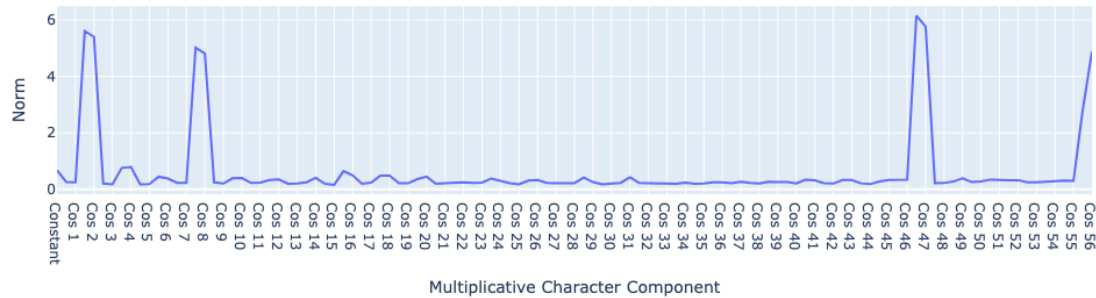
```

In [24]: # Project W_E onto the multiplicative basis
# VISUAL (CRITICAL): Is this spectrum sparse (H1) or dense (H4)?
mult_embed = mult_basis @ W_E # (113, 128)
mult_norms = mult_embed.norm(dim=-1) # (113,)

line(
    mult_norms,
    x=mult_basis_names,
    xaxis="Multiplicative Character Component",
    yaxis="Norm",
    title="Embedding Spectrum in MULTIPLICATIVE Basis – Sparse or Dense?"
)

```

Embedding Spectrum in MULTIPLICATIVE Basis — Sparse or Dense?



```
In [25]: # VISUAL (CRITICAL): Side-by-side comparison – the headline result
# Both spectra plotted with the same y-axis scale for fair comparison
import plotly.graph_objects as go
from plotly.subplots import make_subplots

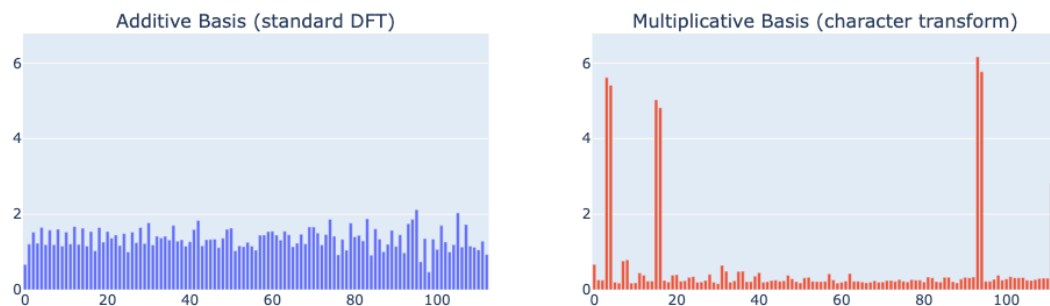
fig = make_subplots(rows=1, cols=2, subplot_titles=["Additive Basis (standard",

add_norms_np = utils.to_numpy(fourier_norms)
mult_norms_np = utils.to_numpy(mult_norms)
y_max = max(add_norms_np.max(), mult_norms_np.max()) * 1.1

fig.add_trace(go.Bar(y=add_norms_np, name="Additive"), row=1, col=1)
fig.add_trace(go.Bar(y=mult_norms_np, name="Multiplicative"), row=1, col=2)

fig.update_yaxes(range=[0, y_max], row=1, col=1)
fig.update_yaxes(range=[0, y_max], row=1, col=2)
fig.update_layout(title_text="Embedding Fourier Spectrum: Additive vs Multip
fig.show()
```

Embedding Fourier Spectrum: Additive vs Multiplicative Basis



15. Quantitative Sparsity Metrics

Visual inspection can be deceiving. We need numbers.

Gini coefficient: measures inequality in a distribution. Gini = 0 means perfectly uniform (all bars equal height). Gini = 1 means all energy in one bar. Higher Gini = sparser spectrum.

Inverse Participation Ratio (IPR): measures concentration. $IPR = 1$ means all energy in one component. $IPR = n$ means perfectly uniform across n components. We report $1/IPR$ as the "effective number of active components." Lower effective dim = sparser.

Both are standard in the grokking literature (Gromov 2023, Nanda et al. 2023).

```
In [26]: # Define reusable sparsity metrics

def gini_coefficient(x):
    """Gini coefficient of a 1D tensor. 0 = uniform, 1 = maximally sparse."""
    x = x.detach().cpu().to(torch.float64).abs()
    x = x.sort().values
    n = len(x)
    index = torch.arange(1, n + 1, dtype=torch.float64)
    return ((2 * index - n - 1) * x).sum() / (n * x.sum())

def inverse_participation_ratio(x):
    """IPR of a 1D tensor. 1 = all energy in one component, n = uniform."""
    x = x.detach().cpu().to(torch.float64).abs()
    x2 = x ** 2
    return (x2.sum() ** 2) / (x2 ** 2).sum() if (x2 ** 2).sum() > 0 else torch.nan

# Sanity checks
uniform = torch.ones(56)
one_hot = torch.zeros(56); one_hot[0] = 1.0
print(f"Sanity checks:")
print(f"  Gini(uniform) = {gini_coefficient(uniform):.4f} (should be ~0)")
print(f"  Gini(one-hot) = {gini_coefficient(one_hot):.4f} (should be ~1)")
print(f"  IPR(uniform) = {inverse_participation_ratio(uniform):.1f} (should be ~56)")
print(f"  IPR(one-hot) = {inverse_participation_ratio(one_hot):.1f} (should be ~1)")
print(f"  1/IPR(uniform) = {1/inverse_participation_ratio(uniform):.4f} (effective dim, should be ~1/56)")
print(f"  1/IPR(one-hot) = {1/inverse_participation_ratio(one_hot):.4f} (effective dim, should be ~1)")
```

Sanity checks:

```
Gini(uniform) = 0.0000 (should be ~0)
Gini(one-hot) = 0.9821 (should be ~1)
IPR(uniform) = 56.0 (should be ~56)
IPR(one-hot) = 1.0 (should be ~1)
1/IPR(uniform) = 0.0179 (effective dim, should be ~1/56)
1/IPR(one-hot) = 1.0000 (effective dim, should be ~1)
```

```
In [27]: # QUANTITATIVE: Compute Gini and IPR for both spectra
# First, combine sin+cos norms per frequency (56 values per basis)

def combined_freq_norms(norms_tensor):
    """From a (113,) norms tensor [constant, sin1, cos1, sin2, cos2, ...],
    return (56,) combined norms: sqrt(sin_k^2 + cos_k^2) for k=1..56."""
    result = []
    for k in range(1, q // 2 + 1):
        sin_n = norms_tensor[2*k - 1].item()
        cos_n = norms_tensor[2*k].item()
        result.append((sin_n**2 + cos_n**2)**0.5)
    return torch.tensor(result)
```

```

additive_freq_norms = combined_freq_norms(fourier_norms)
mult_freq_norms = combined_freq_norms(mult_norms)

# Compute metrics
gini_add = gini_coefficient(additive_freq_norms).item()
gini_mult = gini_coefficient(mult_freq_norms).item()
ipr_add = inverse_participation_ratio(additive_freq_norms).item()
ipr_mult = inverse_participation_ratio(mult_freq_norms).item()

print(f"{'Metric':<30s} {'Additive':>10s} {'Multiplicative':>14s} {'Ratio':>10s}")
print(f"{'-'*70}")
print(f"{'Gini coefficient':<30s} {gini_add:>10.4f} {gini_mult:>14.4f} {gini_mult/gini_add:>10.2f}")
print(f"{'IPR':<30s} {ipr_add:>10.2f} {ipr_mult:>14.2f}")
print(f"{'Effective dim (1/IPR)':<30s} {1/ipr_add:>10.2f} {1/ipr_mult:>14.2f}")
print()
print("Interpretation:")
print(f"  Gini_mult / Gini_add = {gini_mult/max(gini_add,1e-8):.2f}x")
if gini_mult > gini_add * 1.5:
    print("  >>> Multiplicative basis is SPARSER. Supports H1 (discrete-log Clock). <<<")
elif gini_mult < gini_add * 0.67:
    print("  >>> Additive basis is sparser. Unexpected result. <<<")
else:
    print("  >>> Similar sparsity in both bases. Supports H4 (dense Fourier)")

```

Metric	Additive	Multiplicative	Ratio
Gini coefficient	0.0714	0.5794	8.12x
IPR	52.68	4.06	
Effective dim (1/IPR)	0.02	0.25	12.98x

Interpretation:

Gini_mult / Gini_add = 8.12x

>>> Multiplicative basis is SPARSER. Supports H1 (discrete-log Clock). <<<

```

In [28]: # Auto-detect key frequencies in both bases
# Using 5x median threshold (raised from 3x to exclude borderline cases like

def detect_key_freqs(freq_norms, threshold_multiplier=5.0):
    """Given (56,) combined frequency norms, return sorted list of key frequencies
    norms_list = [(k+1, freq_norms[k].item()) for k in range(len(freq_norms))]
    norms_list.sort(key=lambda x: -x[1])
    median_norm = sorted([n for _, n in norms_list])[len(norms_list)//2]
    keys = sorted([k for k, n in norms_list if n > threshold_multiplier * median_norm])
    return keys, median_norm

key_freqs_add, median_add = detect_key_freqs(additive_freq_norms)
key_freqs_mult, median_mult = detect_key_freqs(mult_freq_norms)

print(f"Additive key frequencies: {key_freqs_add}")
print(f" ({len(key_freqs_add)} out of {q//2}, threshold = {5*median_add:.4f})")
print(f"\nMultiplicative key frequencies: {key_freqs_mult}")
print(f" ({len(key_freqs_mult)} out of {q//2}, threshold = {5*median_mult:.4f})")

# Show the gap between key freqs and the next largest
print(f"\nTop 6 multiplicative frequencies by combined norm:")

```

```

sorted_mult = sorted([(k+1, mult_freq_norms[k].item()) for k in range(len(mu
for k, n in sorted_mult[:6]:
    marker = " <-- KEY" if k in key_freqs_mult else ""
    print(f"  freq {k:>3d}: norm = {n:.4f}{marker}")

# Check for CRT subgroup structure (H7)
# 112 = 2^4 * 7. Multiples of 16 -> period 7; multiples of 7 -> period 16
if key_freqs_mult:
    mult_of_7 = [k for k in key_freqs_mult if k % 7 == 0]
    mult_of_16 = [k for k in key_freqs_mult if k % 16 == 0]
    print(f"\nH7 subgroup check:")
    print(f"  Key freqs that are multiples of 7: {mult_of_7}")
    print(f"  Key freqs that are multiples of 16: {mult_of_16}")
    if mult_of_7 or mult_of_16:
        print("  Some CRT-related structure detected!")
    else:
        print("  No obvious CRT subgroup pattern.")

```

Additive key frequencies: []
 (0 out of 56, threshold = 9.7530)

Multiplicative key frequencies: [2, 8, 47, 56]
 (4 out of 56, threshold = 1.8260)

Top 6 multiplicative frequencies by combined norm:

```

freq 47: norm = 8.4447 <-- KEY
freq  2: norm = 7.7959 <-- KEY
freq  8: norm = 6.9641 <-- KEY
freq 56: norm = 5.6402 <-- KEY
freq  4: norm = 1.0966
freq 16: norm = 0.8126

```

H7 subgroup check:

```

Key freqs that are multiples of 7: [56]
Key freqs that are multiples of 16: []
Some CRT-related structure detected!

```

16. SVD of the Embedding

The SVD decomposes W_E into principal components. If the model represents integers via their discrete logs, reordering the SVD components by discrete log should reveal clean sinusoidal waves (just like addition's SVD shows sine waves in integer order).

In [29]:

```

# SVD of embedding matrix
U_svd, S_svd, Vh_svd = torch.linalg.svd(W_E.cpu().to(torch.float32), full_matrices=True)
effective_rank = (S_svd.sum() ** 2 / (S_svd ** 2).sum()).item()

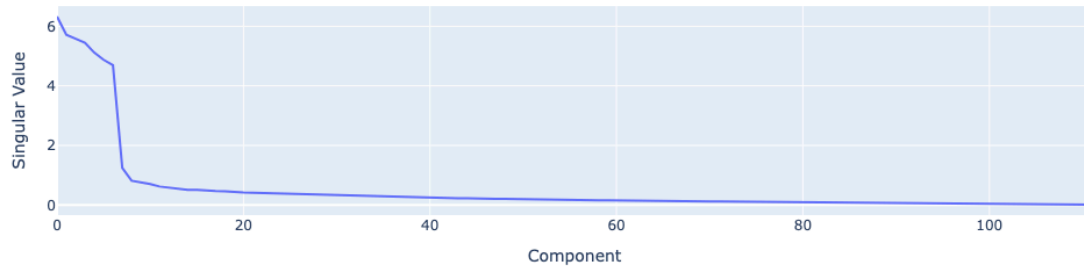
print(f"Effective rank of W_E: {effective_rank:.1f} (out of {min(W_E.shape)})")
line(S_svd, title="Singular Values of W_E (multiplication)", xaxis="Component")

# Top 8 principal components in integer order
line(U_svd[:, :8].T, title="Top 8 Principal Components (integer order)", xaxis="Integer Order")

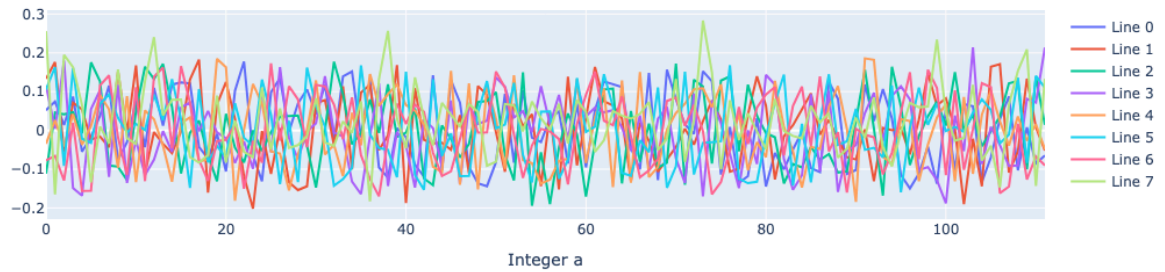
```

Effective rank of W_E : 17.1 (out of 112)

Singular Values of W_E (multiplication)



Top 8 Principal Components (integer order)



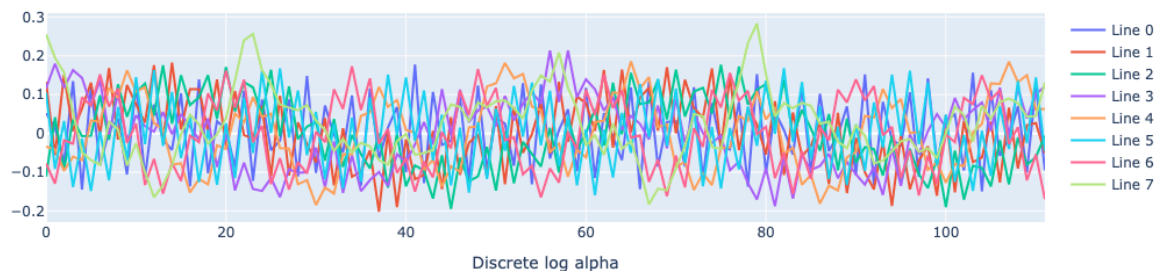
```
In [30]: # VISUAL (CRITICAL): Reorder SVD components by discrete log
# If H1 holds, these should become clean sine waves

# log_order_perm[alpha] = index into W_E rows for the integer with discrete
# W_E rows are 0-indexed for integers 1,...,112, so integer a is at index a-
log_order_perm = [pow(g, alpha, p) - 1 for alpha in range(q)]

U_reordered = U_svd[log_order_perm]

line(U_reordered[:, :8].T, title="Top 8 Principal Components (REORDERED by c
# If H1: smooth periodic waves. If H4: still noisy.
```

Top 8 Principal Components (REORDERED by discrete log)



17. Neuron Heatmaps Reordered by Discrete Log

In Section 12, the neuron heatmaps (112x112 grids in integer order) looked complex. If the model computes in discrete-log space, reordering both axes by `log_g` should reveal clean diagonal stripes, just like addition neurons show in integer order.

This is a visual test independent of the Fourier analysis above.

```
In [31]: # VISUAL (CRITICAL): Reordered neuron heatmaps
# Pick the best neuron for each key frequency (highest fraction at that freq)

# We need mult_neuron_freq_norm from the clustering cell below.
# If it hasn't been computed yet, compute it here.
try:
    _ = mult_neuron_freq_norm.shape
except NameError:
    print("Computing neuron freq fractions first...")
    mult_fourier_acts = mult_basis @ einops.rearrange(neuron_acts, "(a b) n")
    mult_fourier_acts[:, 0, 0] = 0.0
    mult_neuron_freq_norm = torch.zeros(q//2, 512).to(neuron_acts.device)
    for freq in range(q//2):
        for xi in [0, 2*(freq+1)-1, 2*(freq+1)]:
            for yi in [0, 2*(freq+1)-1, 2*(freq+1)]:
                if xi < mult_fourier_acts.shape[1] and yi < mult_fourier_acts.shape[2]:
                    mult_neuron_freq_norm[freq] += mult_fourier_acts[:, xi, yi]
    total_e = mult_fourier_acts.pow(2).sum(dim=[-1, -2])
    mult_neuron_freq_norm = mult_neuron_freq_norm / total_e[None, :].clamp(min=1e-6)

# For each key frequency, find the neuron with the highest fraction
print("Best neuron per key frequency:")
selected_neurons = []
for k in key_freqs_mult:
    best_neuron = mult_neuron_freq_norm[k-1].argmax().item()
    best_frac = mult_neuron_freq_norm[k-1, best_neuron].item()
    print(f" Freq {k}: neuron {best_neuron} ({best_frac*100:.1f}% energy at this freq)")
    selected_neurons.append((best_neuron, k))

# Show each selected neuron: integer order vs discrete-log order
for neuron_idx, freq_k in selected_neurons:
    H = einops.rearrange(neuron_acts[:, neuron_idx], "(a b) -> a b", a=q, b=q)
    H_reordered = H[log_order_perm][:, log_order_perm]

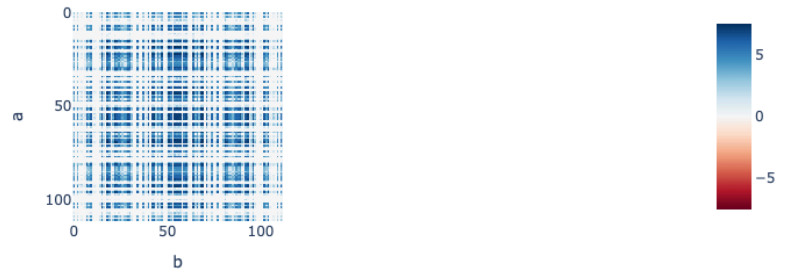
    imshow(H, title=f"Neuron {neuron_idx} (freq {freq_k}) - Integer Order",
            figsize=(10, 10))
    imshow(H_reordered, title=f"Neuron {neuron_idx} (freq {freq_k}) - Discrete-Log Order",
            figsize=(10, 10),
            xaxis="log_g(b)", yaxis="log_g(a)")
```

Computing neuron freq fractions first...

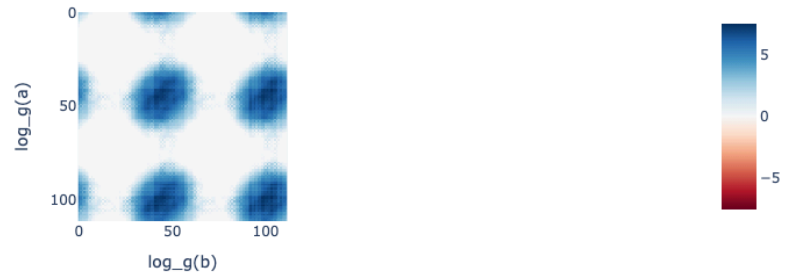
Best neuron per key frequency:

```
Freq 2: neuron 216 (95.5% energy at this freq)
Freq 8: neuron 437 (95.1% energy at this freq)
Freq 47: neuron 344 (96.6% energy at this freq)
Freq 56: neuron 239 (99.4% energy at this freq)
```

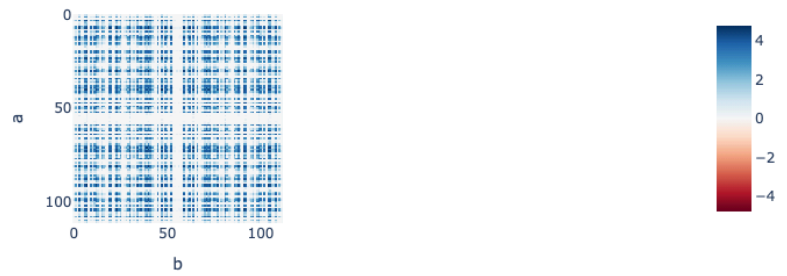
Neuron 216 (freq 2) — Integer Order



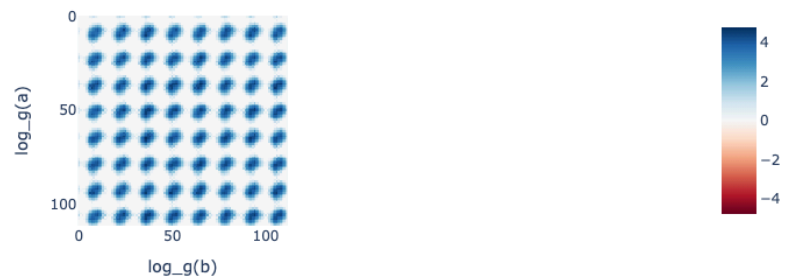
Neuron 216 (freq 2) — Discrete Log Order



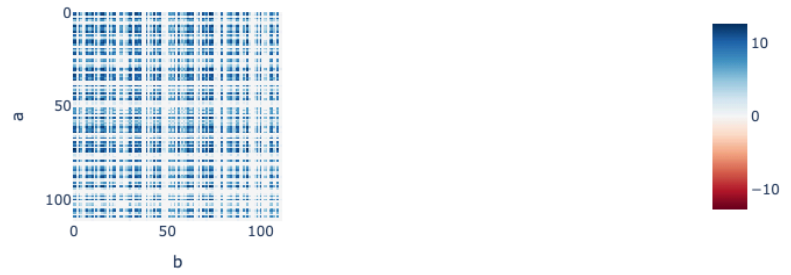
Neuron 437 (freq 8) — Integer Order



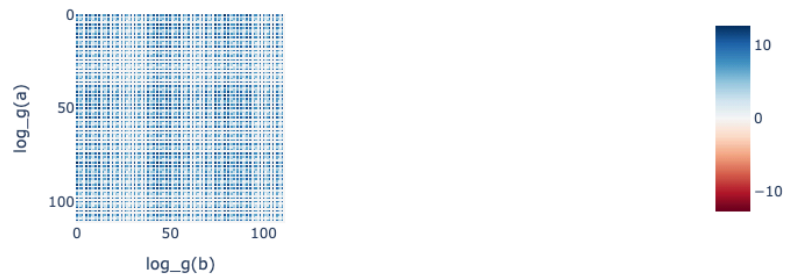
Neuron 437 (freq 8) — Discrete Log Order



Neuron 344 (freq 47) — Integer Order



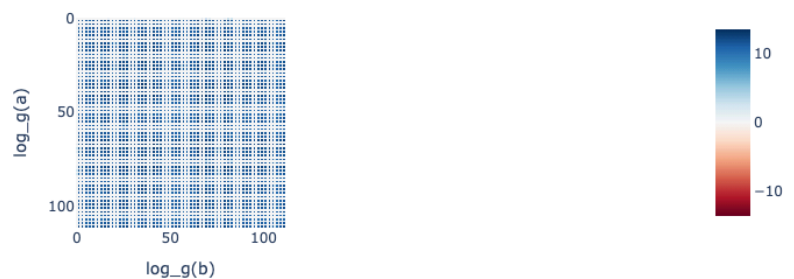
Neuron 344 (freq 47) — Discrete Log Order



Neuron 239 (freq 56) — Integer Order



Neuron 239 (freq 56) — Discrete Log Order



18. Neuron Frequency Clustering: Multiplicative vs Additive

For each of the 512 neurons, we measure what fraction of its 2D Fourier energy is at each frequency, in both bases. If the multiplicative basis is correct, each neuron should be cleanly dominated by a single multiplicative frequency.

```
In [32]: # Compute neuron frequency fractions in BOTH bases
# Following the same pattern as Nanda's notebook

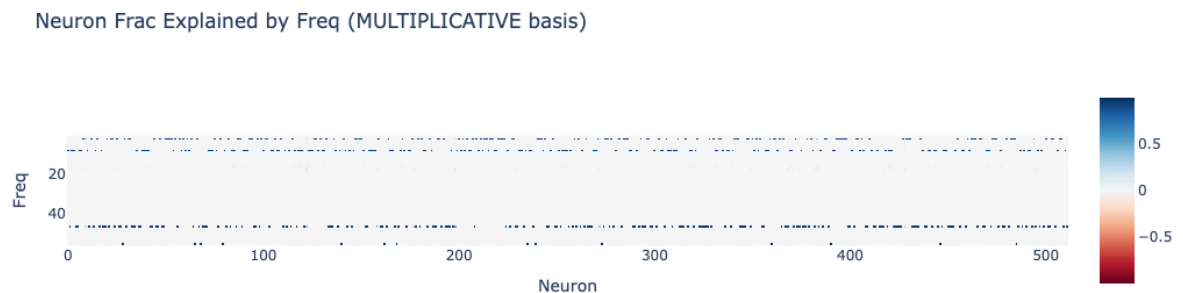
def compute_neuron_freq_fractions(basis, neuron_activations, n_positions, d_
    """Compute fraction of each neuron's 2D Fourier energy at each frequency
    Returns (n_freqs, d_mlp) tensor."""
    n_freqs = n_positions // 2
    # 2D Fourier transform of all neurons at once
    fourier_acts = basis @ einops.rearrange(neuron_activations, "(a b) neuro
    # Zero out DC component
    fourier_acts[:, 0, 0] = 0.0

    # For each frequency k, sum energy in the 3x3 block: {0, 2k-1, 2k} x {0,
    freq_norm = torch.zeros(n_freqs, d_mlp).to(neuron_activations.device)
    for freq in range(n_freqs):
        for x_idx in [0, 2*(freq+1) - 1, 2*(freq+1)]:
            for y_idx in [0, 2*(freq+1) - 1, 2*(freq+1)]:
                if x_idx < fourier_acts.shape[1] and y_idx < fourier_acts.sh
                    freq_norm[freq] += fourier_acts[:, x_idx, y_idx] ** 2

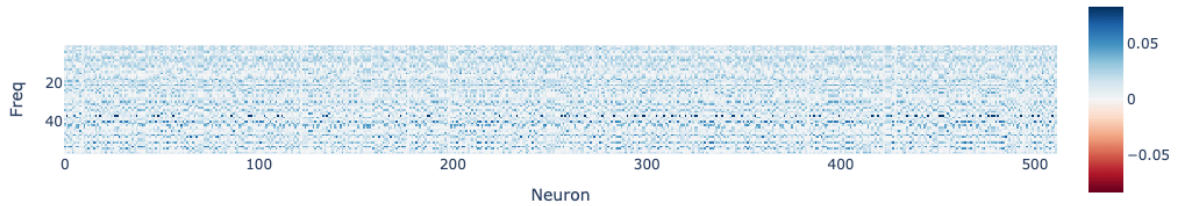
    # Normalize by total energy per neuron
    total_energy = fourier_acts.pow(2).sum(dim=[-1, -2])
    freq_norm = freq_norm / total_energy[None, :].clamp(min=1e-8)
    return freq_norm

# Multiplicative basis
mult_neuron_freq_norm = compute_neuron_freq_fractions(mult_basis, neuron_act
imshow(mult_neuron_freq_norm, xaxis="Neuron", yaxis="Freq",
       y=list(range(1, q//2+1)), title="Neuron Frac Explained by Freq (MULTI

# Additive basis
add_neuron_freq_norm = compute_neuron_freq_fractions(fourier_basis, neuron_a
imshow(add_neuron_freq_norm, xaxis="Neuron", yaxis="Freq",
       y=list(range(1, q//2+1)), title="Neuron Frac Explained by Freq (ADDIT
```



Neuron Frac Explained by Freq (ADDITIVE basis)



```
In [33]: # QUANTITATIVE: Compare neuron clustering quality between bases

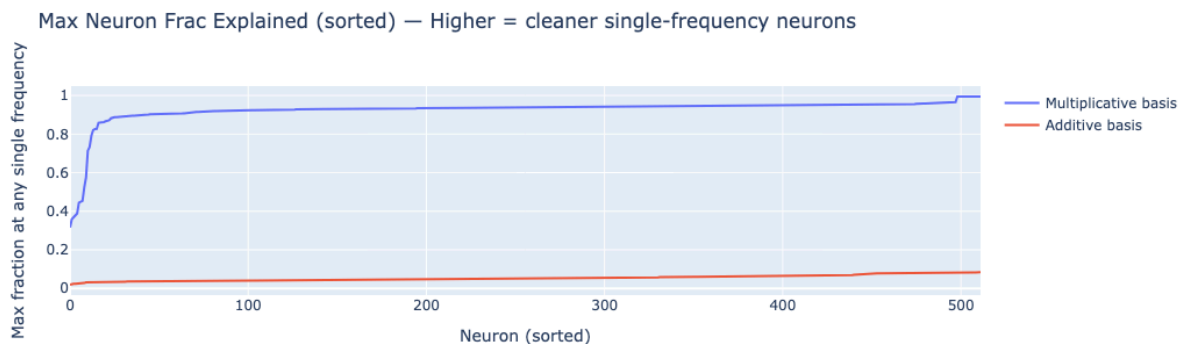
mult_max_frac = mult_neuron_freq_norm.max(dim=0).values # (512,)
add_max_frac = add_neuron_freq_norm.max(dim=0).values # (512,)

# Sorted max-fraction curves
line([mult_max_frac.sort().values, add_max_frac.sort().values],
     line_labels=["Multiplicative basis", "Additive basis"],
     title="Max Neuron Frac Explained (sorted) — Higher = cleaner single-freq",
     xaxis="Neuron (sorted)", yaxis="Max fraction at any single frequency")

# Quantitative summary
threshold = 0.85
n_clean_mult = (mult_max_frac > threshold).sum().item()
n_clean_add = (add_max_frac > threshold).sum().item()

print(f"\nNeurons with >{threshold*100:.0f}% energy at one frequency:")
print(f"  Multiplicative basis: {n_clean_mult} / 512 ({n_clean_mult/512*100:.1f}%)")
print(f"  Additive basis:      {n_clean_add} / 512 ({n_clean_add/512*100:.1f}%)")
print(f"\nMean max-fraction:")
print(f"  Multiplicative: {mult_max_frac.mean().item():.4f}")
print(f"  Additive:      {add_max_frac.mean().item():.4f}")

if n_clean_mult > n_clean_add * 1.5:
    print("\n>>> Multiplicative basis produces cleaner neuron clustering. Success")
elif n_clean_add > n_clean_mult * 1.5:
    print("\n>>> Additive basis produces cleaner clustering. Unexpected for this dataset")
else:
    print("\n>>> Similar clustering quality in both bases. <<<")
```



Neurons with >85% energy at one frequency:

Multiplicative basis: 496 / 512 (96.9%)

Additive basis: 0 / 512 (0.0%)

Mean max-fraction:

Multiplicative: 0.9249

Additive: 0.0538

>>> Multiplicative basis produces cleaner neuron clustering. Supports H1. <<
<

```
In [34]: # === Train Multiplication Models on New Primes ===
# Architecture: 1-layer, d_model=128, d_mlp=512, 4 heads, ReLU (same as p=11)

TRAIN = False # SET TO True TO TRAIN
PRIMES = [59,61,67, 71, 73, 79, 83, 89, 97]
SEED = 43
MAX_EPOCHS = 100000
TRAIN_FRAC = 0.3
LR = 1e-3
WD = 1.0
CHECKPOINT_EVERY = 100
EARLY_STOP_THRESHOLD = 0.01
EARLY_STOP_PATIENCE = 500

if TRAIN:
    for P_NEW in PRIMES:
        Q_NEW = P_NEW - 1
        print(f"\n{' '*60}")
        print(f"Training: a*b mod {P_NEW} | q={Q_NEW}")
        print(f"{' '*60}")

        # Dataset
        a_vec = einops.repeat(torch.arange(1, P_NEW), "i -> (i j)", j=Q_NEW)
        b_vec = einops.repeat(torch.arange(1, P_NEW), "j -> (i j)", i=Q_NEW)
        eq_vec = torch.full((Q_NEW * Q_NEW,), P_NEW, dtype=torch.long)
        dataset_new = torch.stack([a_vec, b_vec, eq_vec], dim=1).to(device)
        labels_new = ((dataset_new[:, 0].long() * dataset_new[:, 1].long())

        torch.manual_seed(598)
        indices_new = torch.randperm(Q_NEW * Q_NEW)
        cutoff_new = int(Q_NEW * Q_NEW * TRAIN_FRAC)
        train_data = dataset_new[indices_new[:cutoff_new]]
        train_labels = labels_new[indices_new[:cutoff_new]]
        test_data = dataset_new[indices_new[cutoff_new:]]
        test_labels = labels_new[indices_new[cutoff_new:]]
        print(f"Dataset: {Q_NEW*Q_NEW} pairs, train={len(train_data)}, test=

        # Model
        cfg_new = HookedTransformerConfig(
            n_layers=1, n_heads=4, d_model=128, d_head=32, d_mlp=512,
            act_fn="relu", normalization_type=None,
            d_vocab=P_NEW + 1, d_vocab_out=P_NEW, n_ctx=3,
            init_weights=True, device=device, seed=SEED,
        )
```

```

model_new = HookedTransformer(cfg_new)
for name, param in model_new.named_parameters():
    if "b_" in name:
        param.requires_grad = False

# Checkpoint paths
ckpt_path = f"./checkpoints/mult_p{P_NEW}_seed{SEED}.pth"
best_path = f"./checkpoints/mult_p{P_NEW}_seed{SEED}_BEST.pth"

# Optimizer
optimizer = torch.optim.AdamW(model_new.parameters(), lr=LR, weight_

# Resume
start_epoch = 0
train_losses, test_losses = [], []
best_joint = float('inf')
grokked = False
grok_epoch = None

import os
if os.path.exists(ckpt_path):
    ckpt = torch.load(ckpt_path, map_location=device, weights_only=F
    model_new.load_state_dict(ckpt["model"])
    optimizer.load_state_dict(ckpt["optimizer"])
    start_epoch = ckpt["epoch"]
    train_losses = ckpt["train_losses"]
    test_losses = ckpt["test_losses"]
    best_joint = ckpt.get("best_joint", float('inf'))
    grokked = ckpt["grokked"]
    grok_epoch = ckpt["grok_epoch"]
    print(f"Resumed from epoch {start_epoch}")
    if grokked:
        print(f" Already grokked at epoch {grok_epoch}. Skipping.")
        continue

# Training loop
epochs_below = 0
for epoch in tqdm.tqdm(range(start_epoch, MAX_EPOCHS), desc=f"p={P_N
    logits = model_new(train_data[:, -1]).to(torch.float32)
    log_probs = logits.log_softmax(dim=-1)
    train_loss = -log_probs.gather(dim=-1, index=train_labels[:, Nor

    train_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    train_losses.append(train_loss.detach().cpu().item())

    with torch.inference_mode():
        test_logits = model_new(test_data[:, -1]).to(torch.float32)
        test_log_probs = test_logits.log_softmax(dim=-1)
        test_loss = -test_log_probs.gather(dim=-1, index=test_labels
        test_losses.append(test_loss.cpu().item())

    if (epoch + 1) % CHECKPOINT_EVERY == 0:
        torch.save({
            "seed": SEED, "epoch": epoch + 1,

```



```

        "grokked": grokked, "grok_epoch": grok_epoch,
        "model": model_new.state_dict(),
        "optimizer": optimizer.state_dict(),
        "train_losses": train_losses, "test_losses": test_losses,
        "best_joint": best_joint,
    }, ckpt_path)
    joint = train_losses[-1] + test_losses[-1]
    if joint < best_joint:
        best_joint = joint
        torch.save({
            "seed": SEED, "epoch": epoch + 1,
            "model": model_new.state_dict(),
            "train_loss": train_losses[-1],
            "test_loss": test_losses[-1],
        }, best_path)

    if not grokked:
        if test_losses[-1] < EARLY_STOP_THRESHOLD:
            epochs_below += 1
            if epochs_below >= EARLY_STOP_PATIENCE:
                grokked = True
                grok_epoch = epoch + 1
                print(f"\n GROKKED at epoch {grok_epoch}!")
                break
        else:
            epochs_below = 0

    if (epoch + 1) % 5000 == 0:
        print(f" Epoch {epoch+1}: train={train_losses[-1]:.4f}, tes

# Final save
torch.save({
    "seed": SEED, "epoch": len(train_losses),
    "grokked": grokked, "grok_epoch": grok_epoch,
    "model": model_new.state_dict(),
    "optimizer": optimizer.state_dict(),
    "train_losses": train_losses, "test_losses": test_losses,
    "best_joint": best_joint,
}, ckpt_path)

if grokked:
    print(f" p={P_NEW}: GROKKED at epoch {grok_epoch}")
else:
    print(f" p={P_NEW}: did not grok after {len(train_losses)} epoch

print(f"\n{'='*60}\nALL DONE\n{'='*60}")
else:
    print(f"TRAIN = False. Set to True to train.")
    print(f"Primes: {PRIMES}, Seed: {SEED}, Max epochs: {MAX_EPOCHS}")

```

TRAIN = False. Set to True to train.

Primes: [59, 61, 67, 71, 73, 79, 83, 89, 97], Seed: 43, Max epochs: 100000

In [37]: # == Embedding Spectrum (Multiplicative Basis) for Newly Trained Models ==

```

import os
import math

```

```

def find_primitive_root(p):
    """Find smallest primitive root of prime p."""
    q = p - 1
    factors = {}
    n = q
    d = 2
    while d * d <= n:
        while n % d == 0:
            factors[d] = factors.get(d, 0) + 1
            n //= d
        d += 1
    if n > 1:
        factors[n] = 1
    prime_factors = list(factors.keys())
    for g in range(2, p):
        if all(pow(g, q // pf, p) != 1 for pf in prime_factors):
            return g

def embedding_spectrum_mult(model_state, P, seed):
    """Compute multiplicative basis spectrum of W_E."""
    Q = P - 1
    G = find_primitive_root(P)

    cfg = HookedTransformerConfig(
        n_layers=1, n_heads=4, d_model=128, d_head=32, d_mlp=512,
        act_fn="relu", normalization_type=None,
        d_vocab=P+1, d_vocab_out=P, n_ctx=3,
        init_weights=True, device=device, seed=seed,
    )
    m = HookedTransformer(cfg)
    m.load_state_dict(model_state)

    W_E = m.embed.W_E[1:P].detach().cpu().float()

    dlog = {}
    val = 1
    for alpha in range(Q):
        dlog[val] = alpha
        val = (val * G) % P
    log_vals = torch.tensor([dlog[a] for a in range(1, P)], dtype=torch.float32)

    n_freqs = Q // 2
    basis = [torch.ones(Q, dtype=torch.float32)]
    for freq in range(1, n_freqs + 1):
        basis.append(torch.sin(2 * torch.pi * freq * log_vals / Q))
        basis.append(torch.cos(2 * torch.pi * freq * log_vals / Q))
    basis = torch.stack(basis, dim=0)
    basis = basis / basis.norm(dim=-1, keepdim=True).clamp(min=1e-8)

    proj = basis @ W_E
    norms = proj.norm(dim=-1)
    freq_norms = []
    for k in range(1, n_freqs + 1):
        s, c = norms[2*k-1].item(), norms[2*k].item()
        freq_norms.append((s**2 + c**2)**0.5)

```

```

    return torch.tensor(freq_norms), G

# Color palette (one per prime)
palette = px.colors.qualitative.G10[:len(PRIMES)]

# Layout: 3 per row
n_cols = 3
n_rows = math.ceil(len(PRIMES) / n_cols)

fig = make_subplots(rows=n_rows, cols=n_cols,
    subplot_titles=[f"p={P_NEW} (q={P_NEW-1})" for P_NEW in PRIMES],
    vertical_spacing=0.12, horizontal_spacing=0.06)

for idx, P_NEW in enumerate(PRIMES):
    row = idx // n_cols + 1
    col = idx % n_cols + 1

    ckpt_path = f"./checkpoints/mult_p{P_NEW}_seed{SEED}.pth"
    if not os.path.exists(ckpt_path):
        print(f"p={P_NEW}: checkpoint not found")
        continue

    ckpt = torch.load(ckpt_path, map_location=device, weights_only=False)
    print(f"p={P_NEW}: epoch {ckpt['epoch']}, grokked={ckpt['grokked']}")

    freq_norms, _ = embedding_spectrum_mult(ckpt["model"], P_NEW, SEED)
    Q_NEW = P_NEW - 1

    fig.add_trace(go.Bar(
        x=list(range(1, Q_NEW//2 + 1)),
        y=freq_norms.tolist(),
        marker_color=palette[idx % len(palette)],
        showlegend=False,
    ), row=row, col=col)
    fig.update_xaxes(title_text="k", row=row, col=col)

fig.update_layout(
    title=dict(text="W_E Spectrum in Multiplicative Basis (all primes, seed=SEED)",
        height=300*n_rows, width=1000,
    )
)
fig.show()

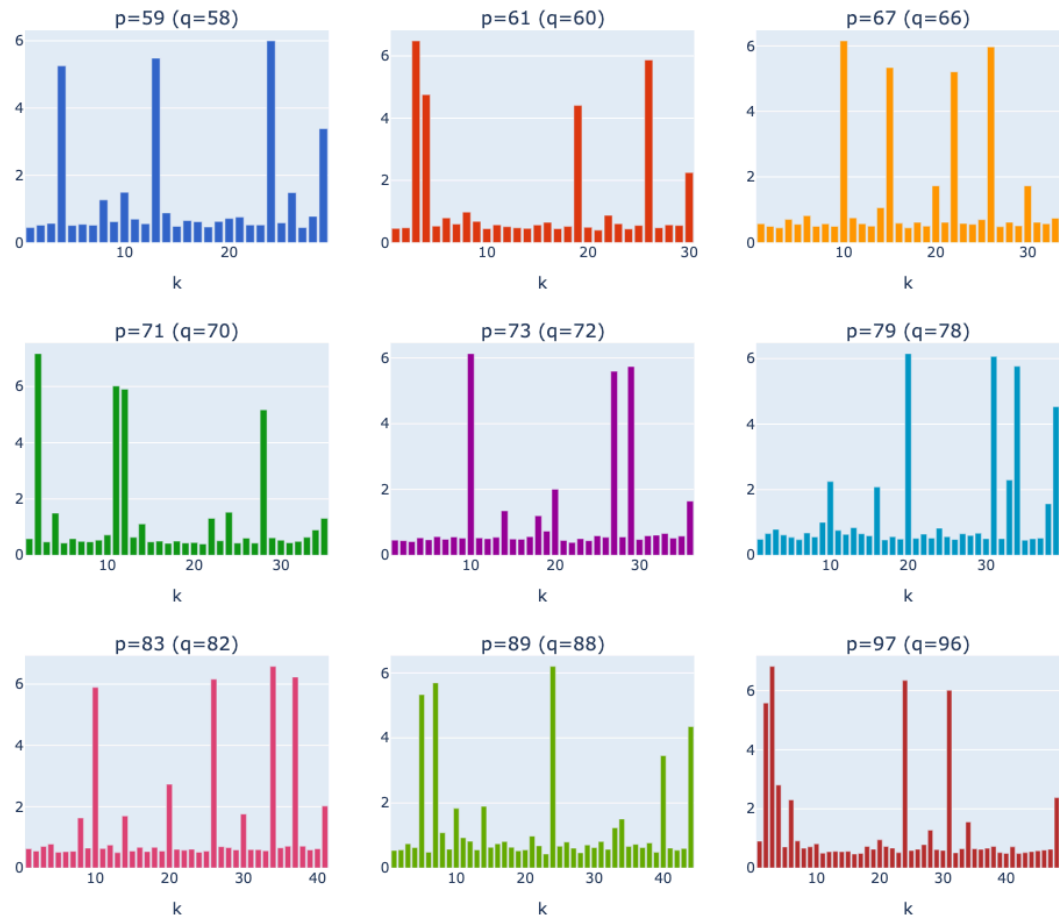
# Print key frequencies
print(f"\n{'Prime':<8} {'q':<6} {'Key freqs (>3x median)':<40} {'Count'}")
print("-" * 65)
for P_NEW in PRIMES:
    ckpt_path = f"./checkpoints/mult_p{P_NEW}_seed{SEED}.pth"
    if not os.path.exists(ckpt_path):
        continue

    ckpt = torch.load(ckpt_path, map_location=device, weights_only=False)
    freq_norms, _ = embedding_spectrum_mult(ckpt["model"], P_NEW, SEED)
    median = freq_norms.median().item()
    key_freqs = [k+1 for k in range(len(freq_norms)) if freq_norms[k].item() > 3*median]
    print(f"p={P_NEW:<5} {P_NEW-1:<6} {str(key_freqs):<40} {len(key_freqs)}")

```

p=59: epoch 14648, grokked=True
p=61: epoch 23519, grokked=True
p=67: epoch 36080, grokked=True
p=71: epoch 23918, grokked=True
p=73: epoch 23358, grokked=True
p=79: epoch 15518, grokked=True
p=83: epoch 14078, grokked=True
p=89: epoch 12960, grokked=True
p=97: epoch 12066, grokked=True

W_E Spectrum in Multiplicative Basis (all primes, seed=43)



Prime	q	Key freqs (>3x median)	Count
p=59	58	[4, 13, 24, 29]	4
p=61	60	[3, 4, 19, 26, 30]	5
p=67	66	[10, 15, 22, 26]	4
p=71	70	[2, 11, 12, 28]	4
p=73	72	[10, 20, 27, 29, 36]	5
p=79	78	[10, 16, 20, 31, 33, 34, 39]	7
p=83	82	[10, 20, 26, 34, 37, 41]	6
p=89	88	[5, 7, 24, 40, 44]	5
p=97	96	[2, 3, 4, 6, 24, 31, 48]	7

In []: